Propuesta Estructurada

FDSI-GP-05 · Denegación de servicio y consumo económico en un asistente académico con RAG

Asignatura	FDSI / SPTI
Código de equipo	FDSI-GP-05
Tema asignado	Denegación de servicio y consumo económico
Integrantes	Mabel Bernal, Juan Pablo Vélez, Esteban Caicedo, Nicolás Prieto
Repositorio / carpeta de trabajo	https://github.com/MabelBernalAmaya/FDSI-gp05-seminario-ciberseguridad-ia

1. Problema de seguridad

Pensamos en un asistente que le responda preguntas a los estudiantes buscando en los documentos del curso (reglamentos, guías, apuntes). Lo primero que se nos ocurrió fue lo más simple: pegarle al LLM todo el material del curso en el prompt. Pero eso no tiene sentido: ni cabe todo en el contexto que acepta el modelo, ni es viable pagar por mandar el mismo material una y otra vez en cada pregunta. Por eso usamos RAG: el sistema busca solo los documentos que tienen que ver con la pregunta y le manda al LLM solo esos, no todo el temario completo.

El problema es que ni nosotros ni el sistema le pusimos límite a esa búsqueda: no hay tope a cuántas preguntas puede hacer un estudiante, ni a cuántos documentos se pueden traer de una sola vez. El activo que queremos cuidar es que el asistente siga funcionando bien para todos, y que la plata que cuesta usar el LLM (se cobra por token) no se dispare. El riesgo es que alguien —a propósito, o simplemente haciendo una pregunta rara tipo "resume todo el curso"— obligue al sistema a traer un montón de documentos de una vez, o mande muchas preguntas seguidas, y con eso tumbe el servicio para los demás o dispare el gasto sin que nadie se dé cuenta a tiempo (esto se conoce como "denial of wallet").

Si esto no se controla, la universidad puede terminar pagando de más por el uso del LLM, o el asistente puede quedar lento o inservible para el resto de estudiantes mientras dure el abuso.

2. Pregunta de investigación

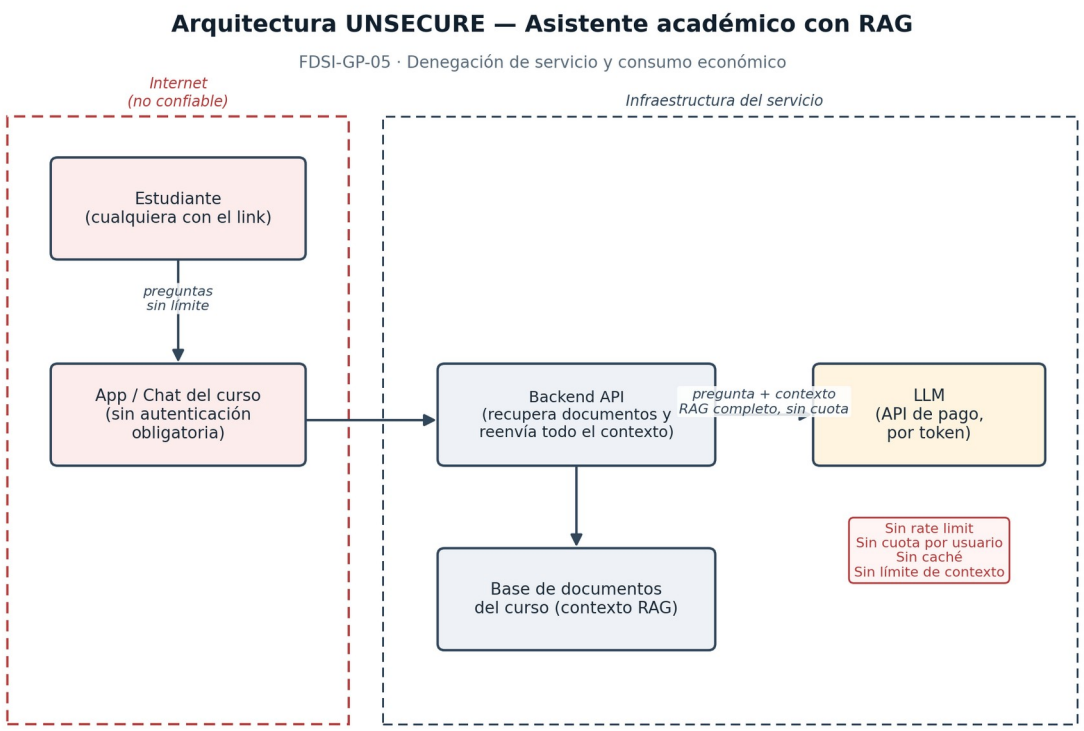
¿Si le ponemos límite de preguntas por usuario, un tope de tokens, caché de respuestas repetidas y un límite a cuántos documentos puede traer el RAG por consulta, baja de verdad el gasto en tokens y la cantidad de peticiones que logran tumbar el asistente, comparado con la misma app sin nada de eso?

3. Hipótesis

Creemos que, con esos controles puestos, el consumo de tokens y las peticiones que logran completarse durante un ataque de flooding van a bajar bastante (calculamos una reducción de al menos 70%), sin que un estudiante que use el asistente de forma normal note ninguna diferencia.

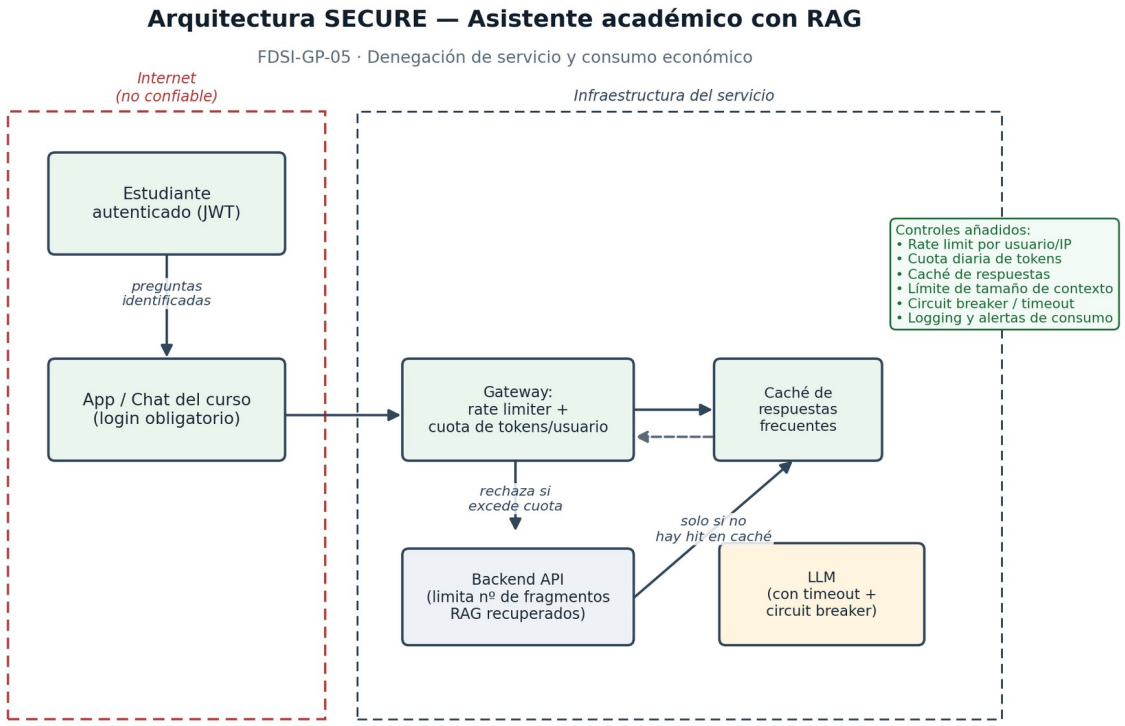
4. Arquitectura Unsecure

Cualquiera puede entrar al chat sin loguearse y preguntar lo que quiera las veces que quiera. Por cada pregunta, el backend busca en los documentos del curso y le manda al LLM todo lo que encontró, sin revisar qué tan grande es eso ni si el estudiante ya preguntó lo mismo antes.



5. Arquitectura Secure

Acá sí toca loguearse. Antes de que la pregunta llegue al LLM pasa por un gateway que revisa cuánta cuota de tokens le queda al estudiante ese día (si ya se pasó, la rechaza ahí mismo sin gastar nada), luego mira si ya hay una respuesta parecida guardada en caché, y si no la hay, el backend limita cuántos documentos puede traer el RAG antes de mandarlo al modelo. También le pusimos un timeout con circuit breaker a la llamada al LLM, por si el proveedor se cae o se demora demasiado. Lo que cambia respecto a la versión de arriba está resaltado en verde en el diagrama.



6. Pruebas propuestas

ID	Prueba	Condición / Entrada	Resultado esperado Unsecure	Resultado esperado Secure
T01	Flooding de preguntas	Un mismo estudiante envía 100 preguntas en 1 minuto con un script	Se procesan las 100 y se consumen tokens sin control	Solo se procesan las que caben en la cuota (ej. 10/min); el resto se rechaza (HTTP 429)
T02	Pregunta que fuerza recuperación masiva de contexto RAG	Preguntar algo que obligue a traer decenas de documentos completos (ej. "resume todos los documentos del curso")	Se recuperan y envían todos los fragmentos sin límite; el contexto se dispara	El backend limita el número de fragmentos/tokens de contexto recuperado por consulta
T03	Repetición de la misma consulta	Varios estudiantes preguntan lo mismo (una FAQ típica) 20 veces en total	El backend llama al LLM las 20 veces	A partir de la 2ª, la respuesta sale de caché sin volver a llamar al LLM
T04	Ráfaga simulada multiusuario	Simular 10 estudiantes/IPs distintos enviando ráfagas al tiempo	El servicio se vuelve lento o deja de responder a estudiantes legítimos	El rate limiter contiene el abuso por usuario/IP; el servicio se mantiene disponible
T05	Estudiante legítimo bajo ataque	Mientras corre T04, un estudiante normal hace una consulta puntual	La consulta tarda mucho o falla (latencia se dispara)	La consulta del estudiante legítimo responde en un tiempo aceptable

7. Criterios de evaluación

Métrica / criterio	Qué mide	Cómo se calcula / observa	Umbral o resultado esperado
Tokens consumidos totales	El costo económico del escenario de prueba	Sumar los tokens reportados por la API del LLM (o contados con tiktoken) en cada corrida	Reducción ≥ 70% en Secure vs. Unsecure
Tasa de solicitudes rechazadas por el rate limiter	Efectividad del control de cuota	(solicitudes rechazadas / solicitudes totales) durante el flooding (T01, T04)	≥ 90% de las solicitudes que exceden la cuota quedan rechazadas
Latencia (p95) de un estudiante legítimo durante el ataque	Disponibilidad real del servicio	Medir p95 de tiempo de respuesta en T05, Unsecure vs. Secure	Latencia en Secure ≤ 2× la latencia normal sin ataque
% de solicitudes repetidas resueltas por caché	Eficacia de la caché para bajar llamadas al LLM	(respuestas servidas desde caché / total de solicitudes repetidas) en T03	≥ 80% de hit rate

8. Tecnologías previstas

Pensamos en usar vLLM para la parte de inferencia porque optimiza bastante el uso de memoria, pero necesita GPU y no tenemos acceso a una, así que lo dejamos por fuera del alcance (queda como trabajo futuro si el proyecto sigue después del curso). Nos quedamos con algo que sí podamos correr en una laptop normal: FastAPI para el backend, LLM vía API (OpenAI, Claude o un modelo local con Ollama, por definir según el acceso que consigamos), y para el RAG una base vectorial ligera (Chroma o FAISS) con embeddings sobre los documentos del curso. Redis para el rate limiter, las cuotas por usuario y la caché de respuestas — ahí guardamos cuántos tokens lleva gastados cada estudiante y las respuestas ya dadas. Tiktoken para contar tokens reales (no solo peticiones). Locust para simular que varios estudiantes preguntan al tiempo. Docker para no complicarnos levantando Redis cada vez.

9. Referencias iniciales

OWASP GenAI Security Project. "LLM10:2025 Unbounded Consumption." https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/

OWASP API Security Top 10 (2023). "API4:2023 Unrestricted Resource Consumption." https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/

Hands-On Architects. "Denial of Wallet: Cost-Aware Rate Limiting for Generative AI Applications." https://handsonarchitects.com/blog/2025/denial-of-wallet-cost-aware-rate-limiting-part-1/

10. Declaración de uso de IA

Usamos Claude (Anthropic) para ayudarnos a organizar la propuesta según la plantilla del curso y a armar un primer borrador de las ideas y los diagramas. Todo lo revisamos, discutimos y ajustamos nosotros como equipo antes de entregar.